
Practicum Report 2

Non-Negative Matrix Factorization

Student: Mohamed Yassine KHALES

May 27, 2026

Introduction

This report describes the design, implementation, and experimental findings of the practical work on **Non-Negative Matrix Factorization (NMF)** with the β -divergence, part of the TSIA 206 course at Telecom Paris. The goal of the practicum was threefold:

1. Derive the *Multiplicative Update Rules* (MUR) from scratch.
2. Implement and validate the algorithm in Python on a real audio signal.
3. Explore the effect of the hyperparameters and reconstruct individual musical notes from a piano recording.

Design and Implementation

Part 1 : Derivation of the MUR algorithm

The derivation of the MUR algorithm gives us the update rule. No problem has been found in this part, we manage to find the update rules :

$$W \leftarrow W \circ \frac{(X \circ \hat{X}^{\beta-2})H^T}{\hat{X}^{\beta-1}H^T} \text{ and } H \leftarrow H \circ \frac{W^T(X \circ \hat{X}^{\beta-2})}{W^T\hat{X}^{\beta-1}}$$

Part 2 : Implementation of the MUR update rule and reconstruction

```
def MUR(X, K, beta, n, print_steps=10):
    eps = 1e-10
    W = np.random.random((X.shape[0], K)) + eps
    H = np.random.random((K, X.shape[1])) + eps
    for i in range(n):
        X_hat = W @ H
        # Update W
        W = W * ((X * X_hat**(beta-2)) @ H.T
                  / (X_hat**(beta-1) @ H.T + eps))
        W += eps
        # Recompute X_hat before updating H
        X_hat = W @ H
        # Update H
        H = H * (W.T @ (X * X_hat**(beta-2))
                  / (W.T @ X_hat**(beta-1) + eps))
        H += eps
        if i % print_steps == 0:
            print(f"Step {i}: divergence = {beta_divergence(X,
                                                              W@H, beta):.4e}")
    return W, H
```

```
def reconstruct(Xstft, W, H):
    y = []
    for i in range(len(Xstft)):
        y.append(Xstft * (W[:,i:i+1]@H[i:i+1,:])/(W@H))
    return np.array(y)
```

Encountered Difficulties

Checking the convergence

Without a convergence criterion, the algorithm runs for a fixed number of iterations regardless of whether the cost has already plateaued. Printing the divergence every few steps (via `print_steps`) was necessary in order to verify if the algorithm is converging.

Dealing with the “nan” problem

The first main problem was the “nan” problem in the convergence verification where the beta divergence gives a “nan” value.

The solution was to add `eps=1e-10` to `X` and `X_hat` in order to avoid division by 0 or log of 0.

```
def beta_divergence(X, X_hat, beta):
    """Calculate the beta divergence.
    - If 'beta=0' : Itakura-Saito (IS) divergence.
    - If 'beta=1' : Kullback-Leibler (KL) divergence."""
    eps = 1e-10
    X = X + eps # <- adding eps
    X_hat = X_hat + eps # <- adding eps
    if beta==0:
        r = X / X_hat
        return np.sum(r-np.log(r)-1)
    if beta ==1:
        return np.sum(X * np.log((X) / (X_hat)) - X + X_hat)
    return np.sum(X**beta / (beta * (beta - 1)) - X * X_hat** (
        beta - 1) / (beta - 1) + X_hat**beta / beta)
```

Memory runout

I tried with a huge value of K , $K = 60$ and I never managed to get the results because of the memory runout. 60 elements is too much for the computer or the google collab to handle. This made me to use values of K that are close to 10 which gave immediate results.